

UD1.Sistema de ficheros e directorios

DAM2-Acceso a Datos 2026-27

0. Java I/O	2
0.1. Sistema de Archivos	2
Directorio raíz	2
Rutas	3
1. La clase File	5
1. Clases para trabajar con ficheros	5
2. Creación de un Objeto File	5
Constructores de la clase File	6
Campos de la clase File	7
3. El objeto File vs. archivo real existente	7
4. Trabajando con un Objeto File	8
5. Métodos más importantes de java.io.File	8
Ejercicios	12
2. Paquete java.nio.file	13
La interface Path	14
Creación de Path	15
Métodos comunes de Path	18
Argumentos Opcionales en métodos de NIO.2	19
La clase Files	21
Conversión entre java.io.File y java.nio.file	22
Mapeo de la Funcionalidad de java.io.File a java.nio.file	22
3. E/S estándar	25
4. Flujos de Datos de E/S	28
Tipos de flujos	28
Flujo de entrada	28
Flujo de salida	29
Flujos de bytes	29
Flujos de caracteres	29
Usando flujos de datos	30
Flujos binarios	32
Flujos de texto	33
Flujos con Búfer	33
Flujos de objetos serializados	34
Flujos desde una URL	35
Anexo. Git	37
Configuración inicial de Git	37

0. Java I/O

Las aplicaciones Java, ¿qué pueden hacer fuera del ámbito de gestionar objetos y atributos en la memoria? ¡Al cerrar el programa se pierde todo! **¿Cómo pueden guardar datos para que la información no se pierda cada vez que el programa se termina? ¡Usar archivos, por supuesto!**, es la primera opción (o cualquier sistema de persistencia más avanzado, como bases de datos, que abordaremos en la siguiente unidad).

Se pueden realizar **programas sencillos que guarden el estado actual de una aplicación en un archivo cada vez que la aplicación se cierra y luego cargue los datos cuando se ejecute la aplicación la próxima vez**. De esta manera, la información se preserva entre ejecuciones del programa. Es lo que se denomina, **persistencia**.

Este apartado estudiaremos el **API `java.io`** para interactuar con archivos y flujos. Comenzamos describiendo **cómo se organizan los archivos y directorios en un sistema de archivos** y mostramos cómo acceder a ellos con la clase **`java.io.File`** (el modo tradicional de hacerlo). Luego veremos **cómo leer y escribir datos de archivo con las clases de flujo** (Streams IO, no confundir con la **`API Streams`**). Concluimos discutiendo formas de leer la entrada del usuario en tiempo de ejecución utilizando la clase **`Console`**.

En el siguiente apartado, dedicado a “**Java NIO.2**”, veremos **cómo Java proporciona técnicas más poderosas (y rápidas) para gestionar archivos**.

0.1. Sistema de Archivos

Para empezar es necesario saber **qué es un sistema de archivos**. Los datos se almacenan en dispositivos de almacenamiento persistentes, **como discos duros o tarjetas de memoria**, por ejemplo.

Un ***archivo*** es un registro dentro del dispositivo de almacenamiento que **contiene datos**.

Los archivos se organizan en ***jerarquías*** utilizando directorios.

Un ***directorio*** es una **ubicación que puede contener archivos y otros directorios**.

Cuando trabajamos con **directorios en Java**, a menudo los tratamos como archivos. De hecho, **se usan muchas de las mismas clases para operar en archivos y directorios**. Por ejemplo, un archivo y un directorio pueden renombrarse con el mismo método de Java.

Para interactuar con archivos, necesitamos conectarnos al ***sistema de archivos***. El ***sistema de archivos*** se encarga de **leer y escribir datos en un ordenador**. Los diferentes sistemas operativos utilizan sistemas de archivos diferentes para gestionar sus datos. Por ejemplo, los sistemas basados en Windows usan un sistema de archivos diferente que los basados en Unix (Linux, ...). La JVM se conectará automáticamente al sistema de archivos local, lo que te permite realizar las mismas operaciones en múltiples plataformas.

Directorio raíz

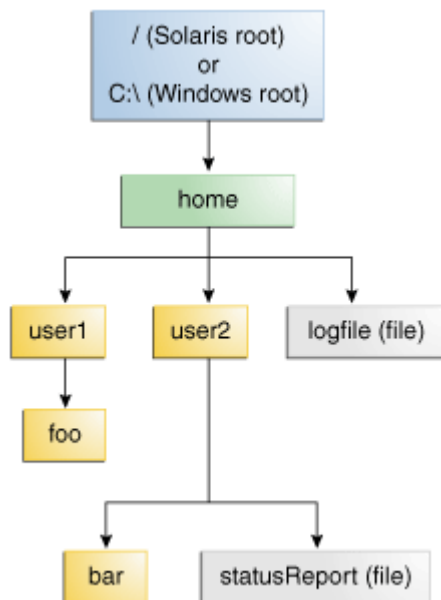
El **directorio raíz (root)** es el directorio superior en el sistema de archivos, del cual heredan todos los archivos y directorios:

- En **Windows**, se denota con una **letra de unidad**, como `c:\\`.
- En **Linux** se denota con una **barra diagonal simple**, `/`.

Rutas

Una **ruta** es una **representación en cadena de un archivo o directorio dentro de un sistema de archivos**. Cada sistema de archivos define su propio carácter separador de rutas que se utiliza entre las entradas de directorio. El valor a la izquierda de un separador es el padre del valor a la derecha del separador. Por ejemplo, el valor de ruta `/home/user/cole.txt` significa que el archivo `cole.txt` está dentro del directorio `user`, con el directorio `user` dentro del directorio `home`.

Las rutas pueden ser absolutas o relativas.



Rreferencia: <https://docs.oracle.com/javase/tutorial/essential/io/path.html>

En la figura anterior muestra un árbol de directorios de ejemplo que contiene un único nodo raíz. Microsoft Windows admite varios nodos raíz. La familia de sistemas operativos basados en Unix (Linux, Solaris, macOS, etc.) admite un único nodo raíz, que se indica mediante el carácter de barra diagonal.

Un archivo se identifica por su ruta en el sistema de ficheros, empezando por el nodo raíz. Por ejemplo, en el sistema de ficheros de Windows, la ruta `C:\Programas\holamundo.kt` identifica un archivo llamado `holamundo.kt` que se encuentra en el directorio `Programas` en la unidad `C:`.

En la figura: `/home/sally/statusReport` y `c:\home\sally\statusReport` son rutas absolutas pa SO Unix y Windows, respectivamente.

El **delimitador** es específico del sistema de archivos. En Linux \ y en Windows /.

Las rutas absolutas y relativas pueden contener símbolos de ruta. Un **símbolo de ruta** es una serie reservada de caracteres que tienen un significado especial dentro de algunos sistemas de archivos. Hay dos símbolos básicos:

Símbolos	Descripción
.	Referencia al directorio actual
..	Referencia al directorio padre del directorio actual

1. La clase File

1. Clases para trabajar con ficheros

La clase `java.io.File` es una de las más empleadas (y antigua) del API `java.io`.

La clase `File` se utiliza para **leer información sobre archivos y directorios existentes, listar el contenido de un directorio o crear/eliminar archivos y directorios**.

Una **instancia de una clase `File`** representa la **ruta a un archivo o directorio específico en el sistema de archivos**, pero **no contiene los datos del archivo o directorio** (el archivo podría no existir).

La clase `File` **no puede leer ni escribir datos dentro de un archivo, aunque se puede pasar como referencia a muchas clases de flujos (y métodos) para leer o escribir datos**. Para escribir leer datos de un archivo, se utilizan las clases de flujo de E/S:

`FileInputStream`, `FileOutputStream`, `FileReader`, `FileWriter`, `RandomAccessFile`, etc.

Por ello, se usa para **convertir el nombre de un archivo y pasarlo como parámetro a otros métodos o constructores que sí pueden leer o escribir datos**.

2. Creación de un Objeto File

Un objeto `File` a menudo **se inicializa con una cadena que contiene una ruta absoluta o relativa al archivo o directorio en el sistema de archivos**.

La **ruta absoluta** de un archivo o directorio es la **ruta completa desde el directorio raíz hasta el archivo o directorio**, incluyendo todos los subdirectorios que contienen el archivo o directorio.

La **ruta relativa** de un archivo o directorio es la **ruta desde el directorio de trabajo actual hasta el archivo o directorio**. Por ejemplo, lo siguiente es una ruta absoluta al archivo `javaio.txt`:

```
/home/usuarioa/apuntes/javaio.txt
```

Lo siguiente es una ruta relativa al mismo archivo, asumiendo que el directorio actual del usuario está configurado en `/home/user`:

```
apuntes/javaio.txt
```

Los sistemas operativos diferentes varían en su formato de nombres de ruta. Por ejemplo, los sistemas basados en Unix usan la barra diagonal hacia adelante, `/`, para las rutas, mientras que los sistemas basados en Windows usan el carácter diagonal inversa, `\`, como separador de ruta.

Muchos lenguajes de programación y sistemas de archivos admiten ambos tipos de barras al escribir declaraciones de ruta. Por conveniencia, **Java ofrece dos opciones para recuperar el carácter separador local: una propiedad del sistema y una variable estática definida en la clase File**. Ambos ejemplos imprimirán el carácter separador para el entorno actual:

```
System.out.println(System.getProperty("file.separator"));

System.out.println(java.io.File.separator);
```

El siguiente código **crea un objeto *File*** y determina si la ruta a la que hace referencia el archivo existe en el sistema de archivos:

```
File javaFile = new File("prueba.txt");

System.out.println(javaFile.exists()); // true, si el archivo existe
```

Este ejemplo proporciona la ruta absoluta a un archivo y muestra true o false según si el archivo existe.

Constructores de la clase File

Constructor	Descripción
File(String pathname)	Crea un objeto File a partir de una ruta en forma de cadena.
File(File parent, String child)	Crea un objeto File a partir de una ruta principal y una secundaria.
File(String parent, String child)	Crea un objeto File a partir de una ruta principal y una secundaria.
File(URI uri)	Crea un objeto File a partir de un URI.

El primero crea un objeto File a partir de una ruta en forma de cadena. Los otros dos constructores se utilizan para crear un objeto File a partir de una ruta principal y una secundaria, como la siguiente:

```
File fichero1 = new File("DATOS", "ud1/prueba.txt");

File directorioPadre = new File("DATOS");
```

```
File fichero2 = new File(directorioPadre, "ud1/prueba.txt");
```

En este ejemplo, creamos dos nuevas instancias de File que son equivalentes la instancia anterior de apuntesJavaIO. **Si la instancia principal es nula, se omitiría y el método volvería al constructor de cadena única.**

Campos de la clase File

La clase File tiene varios campos que puedes usar para acceder a información sobre el sistema de archivos subyacente. Algunos de los campos más útiles son:

Campo	Descripción
static String pathSeparator	El separador de PATH de la plataforma. Por ejemplo, en Windows es ; y en Unix es .
static char pathSeparatorChar	El separador de ruta de la plataforma como un carácter.
static String separator	El separador de ruta de la plataforma. Por ejemplo, en Windows es \ y en Unix es /.
static char separatorChar	El separador de ruta de la plataforma como un carácter.

3. El objeto *File* vs. archivo real existente

Al trabajar con una instancia de la clase File, ten en cuenta que **sólo representa una ruta a un archivo**. A menos que se opere sobre él, **no está conectado a un archivo real en el sistema de archivos**.

Por ejemplo:

- Se puede crear un nuevo objeto File para **comprobar si un archivo existe en el sistema**.
- Se puede llamar a varios métodos para **leer propiedades de archivos dentro del sistema de archivos**.
- Tiene **hay métodos para modificar el nombre o la ubicación de un archivo, así como para eliminarlo**.

La JVM y el sistema de archivos subyacente leerán o modificarán el archivo utilizando los métodos que llamas en la clase File. Si intentas operar en un archivo que no existe o al que

no tienes acceso, **algunos métodos de File lanzarán una excepción**. Otros métodos devolverán false si el archivo no existe o la operación no se puede realizar.

4. Trabajando con un Objeto File

La clase File contiene **numerosos métodos útiles** para interactuar con archivos y directorios en el sistema de archivos. En la siguiente tabla se muestran los más importantes, por su uso:

5. Métodos más importantes de java.io.File

Nombre del Método	Descripción
boolean delete()	Borra el archivo o directorio y devuelve true sólo si la operación se completó con éxito. Si esta instancia es un directorio, el directorio debe estar vacío para poder eliminarse.
boolean exists()	Comprueba si un archivo existe
String getAbsolutePath()	Obtiene el nombre absoluto del archivo o directorio en el sistema de archivos
String getName()	Obtiene el nombre del archivo o directorio
String getParent()	Obtiene el directorio principal en el que se encuentra la ruta, o null si no hay ninguno
boolean isDirectory()	Comprueba si una referencia File es un directorio en el sistema de archivos
boolean isFile()	Comprueba si una referencia File es un archivo en el sistema de archivos
long lastModified()	Devuelve el número de milisegundos desde la época (número de milisegundos desde las 12 a.m. UTC del 1 de enero de 1970) en que se modificó el archivo por última vez
long length()	Obtiene el número de bytes en el archivo
File[] listFiles()	Obtiene una lista de archivos dentro de un directorio

<code>boolean mkdir()</code>	Crea el directorio especificado en la ruta
<code>boolean mkdirs()</code>	Crea el directorio especificado en la ruta, incluyendo cualquier directorio principal inexistente
<code>boolean renameTo(File dest)</code>	Cambia el nombre del archivo o directorio denotado por esta ruta a dest y devuelve true sólo si la operación tuvo éxito.

El siguiente programa de ejemplo muestra información sobre un archivo o directorio, como si existe, qué archivos contiene y más:

```
import java.io.*;

import static java.lang.System.out;

public class Ejemplo1 {

    public static void main(String args[]) throws IOException {

        out.print("Raíz del sistema de ficheros");
        for (File raiz : File.listRoots()) {
            out.format("%s ", raiz);
        }

        out.println();

        for (String nome : args) {

            out.format("%n-----%new File(%s)%n", nome);

            File f = new File(nome);

            out.format("toString(): %s%n", f);
            out.format("exists(): %b%n", f.exists());
            out.format("lastModified(): %tc%n", f.lastModified());
            out.format("isFile(): %b%n", f.isFile());
            out.format("isDirectory(): %b%n", f.isDirectory());
            out.format("isHidden(): %b%n", f.isHidden());
            out.format("canRead(): %b%n", f.canRead());
            out.format("canWrite(): %b%n", f.canWrite());
            out.format("canExecute(): %b%n", f.canExecute());
            out.format("isAbsolute(): %b%n", f.isAbsolute());
            out.format("length(): %d%n", f.length());
        }
    }
}
```

```

        out.format("getName(): %s%n", f.getName());
        out.format("getPath(): %s%n", f.getPath());
        out.format("getAbsolutePath(): %s%n", f.getAbsolutePath());
        out.format("getCanonicalPath(): %s%n", f.getCanonicalPath());
        out.format("getParent(): %s%n", f.getParent());
        out.format("toURI: %s%n", f.toURI());
    }
}
}

```

Ejecutar un programa de consola en vscode con argumentos:

- Menú: Run > Add configurations...
Crea un fichero de configuración launch.json, si no existe.
- Añade los argumentos con la siguiente configuración bajo mainClass...:

```
"args": ["src", "README.md"]
```

El siguiente programa de ejemplo muestra **información sobre el archivo o directorio**, si existe, qué archivos contiene y más:

```

import java.io.File;

public class Ejemplo2 {
    public static void main(String[] args) {
        var archivo = new File("c:\\users");
        System.out.println("Archivo existe: " + archivo.exists());
        if (archivo.exists()) {
            System.out.println("Ruta absoluta: " + archivo.getAbsolutePath());
            System.out.println("Es un directorio: " + archivo.isDirectory());
            System.out.println("Ruta padre: " + archivo.getParent());
            if (archivo.isFile()) {
                System.out.println("Tamaño: " + archivo.length());
                System.out.println("Última modificación: " +
archivo.lastModified());
            } else {
                for (File subArchivo : archivo.listFiles()) {
                    System.out.println(" " + subArchivo.getName());
                }
            }
        }
    }
}

```

En estos ejemplos, ves que la salida de un programa basado en Entrada/Salida **depende por completo de los directorios y archivos disponibles en tiempo de ejecución en el sistema de archivos subyacente.**

Directorio o archivo

Ojo, una ruta podría ser un archivo o un directorio, incluso si tiene una extensión de archivo. ¡No asumas que es uno u otro a menos que lo puedas comprobar! (por ejemplo, .git)

Ejercicios

Debes trabajar únicamente con métodos de la clase [File](#).

Ejercicio 1. Creación y lectura de archivos con File

Realiza los siguientes pasos:

1. Crea un archivo de texto llamado prueba.txt en el directorio actual de tu proyecto, sólo si no existe.
2. Escribe un programa que cree un objeto File para el archivo prueba.txt y compruebe si el archivo existe.
3. Si el archivo existe, muestra la ruta absoluta, nombre del archivo, tamaño, última modificación y si es un directorio.
4. Si el archivo no existe, muestra un mensaje que lo indique y crea uno temporal.

Ejercicio 2. Mostrar el contenido de un directorio

El programa permite la selección de un directorio ([JFileChooser](#) o desde teclado si recoge un parámetro) y usando el método **listFiles()** de la clase File, muestra el contenido de ese directorio, indicando el tamaño de los archivos que contiene y si es un directorio o no. Además, muestra el tamaño total de los archivos y directorios.

Ejercicio 3. Gestor de archivos y directorios

Escribe un programa en Java que funcione como un gestor básico de archivos y directorios. El programa debe permitir al usuario realizar las siguientes operaciones:

1. Crear un directorio, empleando la clase [JFileChooser](#) para seleccionar la ruta donde se creará.
2. Listar todos los archivos y subdirectorios de un directorio de forma recursiva.
3. Eliminar un archivo o directorio. Si es un directorio, eliminar todo su contenido de forma recursiva.
4. Mover o renombrar archivos y directorios.

El programa debe ofrecer un menú para que el usuario elija la operación que desea realizar. La selección de directorios o archivos puede realizarse con la clase [JFileChooser](#).

2. Paquete `java.nio.file`

Antes del lanzamiento de **Java SE 7**, la clase [`java.io.File`](#) era el mecanismo utilizado para la E/S de archivos, pero presentaba varios inconvenientes:

- **Muchos métodos no lanzaban excepciones al fallar**, por lo que era imposible obtener un mensaje de error útil. Por ejemplo, si fallaba la eliminación de un archivo, el programa recibía un “fallo al eliminar”, pero no sabía si era porque el archivo no existía, el usuario no tenía permisos, o había algún otro problema.
- El método `rename` **no funcionaba de manera consistente en diferentes plataformas**.
- **No había un soporte real para enlaces simbólicos**.
- Se **requería más soporte para metadatos**, como permisos de archivos, propietario del archivo y otros atributos de seguridad.
- El **acceso a los metadatos de los archivos era ineficiente**.
- **Muchos de los métodos no escalaban bien**. Solicitar un listado de directorios grandes en un servidor podía causar bloqueos. Los directorios grandes también podían generar problemas de recursos de memoria, lo que resultaba en una denegación de servicio.
- **No era posible escribir código fiable que pudiera recorrer un árbol de archivos recursivamente** y responder adecuadamente si había enlaces simbólicos circulares.

Aun así, **existe mucho código que usa `java.io.File` y sigue siendo útil para muchas situaciones**.

Java SE 7 introdujo una nueva API de E/S de archivos en el paquete [`java.nio.file`](#) que proporciona una funcionalidad más moderna y flexible para manejar archivos y directorios, con mejor rendimiento y opciones avanzadas de gestión de rutas, atributos y recorridos de árbol. Facilita:

- Acceder a rutas y metadatos.
- Leer y escribir archivos.
- Manipular directorios.
- Copiar, mover o borrar archivos y directorios.
- Usar streams y walks para recorrer directorios.

Recursos principales:

Path	Representa una ruta de archivo o directorio (relativa o absoluta). Sustituye a <code>File</code> en muchos casos.	- <code>getFileName()</code> → nombre del archivo. - <code>getParent()</code> → directorio padre. - <code>toAbsolutePath()</code> → convierte en ruta absoluta. - <code>resolve(ruta)</code> → combina rutas. Se obtiene una nueva ruta que resulta de añadir el segundo <code>Path</code> al primero - <code>relativize(ruta)</code> → ruta relativa entre dos rutas. cuál es el camino desde una ruta de origen hasta una ruta de destino - <code>normalize()</code> → limpia las rutas eliminando referencias innecesarias como <code>"."</code> (que indica el directorio actual) o <code>".."</code>
-------------	---	---

Paths	Clase de utilidades para crear objetos Path	- get(String ruta) → crea un Path a partir de una cadena. - get(URI uri) → crea un Path desde una URI.
Files	Contiene métodos estáticos para trabajar con archivos y directorios.	- exists(path), isDirectory(path), size(path). - createFile(path), createDirectory(path), createDirectories(path). - copy(origen, destino, opciones), move(origen, destino, opciones), delete(path). - newDirectoryStream(path) → listado. - walk(path) → recorrido recursivo. - readAllLines(path) → Lee todo el contenido de un archivo de texto - write(path, datos) → Escribe en el archivo las cadenas proporcionadas (sobrescribe si ya existe)
FileSystem	Representa el sistema de archivos. Permite acceder a las raíces (C:, /, etc.)	- FileSystems.getDefault() → sistema de archivos actual. - getRootDirectories() → raíces del sistema. - getPath(String ruta) → crea un Path en este sistema.
DirectoryStream	Permite recorrer el contenido de un directorio de forma eficiente.	- Iteración con for (Path p : stream). - Acepta filtros (newDirectoryStream(path, "*.txt")).
FileVisitor<T>	Interfaz para recorrer directorios recursivamente.	- Métodos: preVisitDirectory, visitFile, postVisitDirectory, visitFileFailed. - Se usa con Files.walkFileTree(path, visitor).
SimpleFileVisitor <T>	Implementación base de FileVisitor. Se puede sobrescribir.	Ejemplo: sobrescribir visitFile() para listar archivos recursivamente.
.attribute. BasicFileAttributes	Contiene metadatos de un archivo (tamaño, fechas, tipo, etc.).	- size() → tamaño en bytes. - creationTime(), lastModifiedTime(). - isDirectory(), isRegularFile().
StandardCopyOption	Enum con opciones para copiar/mover archivos.	- REPLACE_EXISTING → sobrescribir si ya existe. - COPY_ATTRIBUTES → copiar metadatos. - ATOMIC_MOVE → movimiento atómico.

En este apartado, nos centraremos en la **API de la versión 2 de java.nio, o NIO.2 de manera resumida, para interactuar con archivos**. NIO.2 es un acrónimo que significa la segunda versión de la API de **Entrada/Salida No Bloqueante**, y a veces se conoce como la “New I/O.”

NIO.2 nos **permite hacer mucho más con archivos y directorios que la API original de java.io**. Veremos cómo aplicar la API de Streams (ojo, no confundir con “streams” de entrada/salida) para realizar operaciones complejas con archivos y directorios.

La interface Path

La piedra angular de NIO.2 es la interfaz `java.nio.file.Path`. Una instancia de `Path` representa una ruta jerárquica en el sistema de almacenamiento hacia un archivo o directorio. Se puede pensar en un `Path` como el **sustitución de NIO.2 para la clase `java.io.File`**, aunque la forma en que se utiliza es un poco diferente.

Antes de profundizar en eso, hablemos de las similitudes entre estas dos implementaciones. Tanto los objetos `java.io.File` como `Path` pueden hacer referencia a una **ruta absoluta o relativa** dentro del sistema de archivos. Además, ambos pueden hacer referencia a un **archivo o un directorio**.

A diferencia de la clase `java.io.File`, **la interfaz `Path` da soporte para enlaces simbólicos**. Un *enlace simbólico* es un archivo especial dentro de un sistema de archivos que sirve como una referencia o puntero a otro archivo o directorio.

Creación de Path

¿Por qué `Path` es una interface? Cuando se crea un `Path`, la máquina virtual de Java devuelve la implementación específica para el sistema de archivos subyacente. Por ejemplo, la ruta no es igual para Linux que para Windows.

En la mayoría de las circunstancias se desea realizar las mismas operaciones con el `Path`, independientemente del sistema de archivos.

Dado que `Path` es una interfaz, no podemos crear una instancia directamente. Para crearlas se usa el [patrón de diseño Factory](#).

`Path.of()`

La forma más simple y directa de obtener un objeto `Path` es utilizar el método Factory estático [Path.of\(\)](#) definido dentro de la interfaz `Path`. Es fácil crear instancias de `Path` a partir de valores de `String`:

```
Path path1 = Path.of("fotos/batman.png");
Path path2 = Path.of("c:\\users\\usuarioa\\notas.txt");
Path path3 = Path.of("/home/usuarioa");
```

El primer ejemplo crea una referencia a una ruta relativa en el directorio de trabajo actual. El segundo ejemplo crea una referencia a una ruta de archivo absoluta en un sistema basado en Windows. El tercer ejemplo crea una referencia a una ruta de directorio absoluta en un sistema basado en Linux o Mac.

El método `Path.of()` también **incluye varargs (argumentos variables)** para pasar elementos de ruta adicionales. Los valores se combinarán y se separarán automáticamente por el separador de archivos dependiente del sistema operativo que aprendiste en el apartado anterior.

Sobre varargs...  [JAVA: Varargs](#)  DAM - DAW

```
Path path1b = Path.of("fotos", "batman.png");
Path path2b = Path.of("c:", "users", "usuarioa", "notas.txt");
Path path3b = Path.of("/", "home", "usuarioa");
```

Estos ejemplos son simplemente otro modo de escribir los ejemplos anteriores de `Path`, utilizando la lista de parámetros de valores `String` en lugar de un solo valor `String`. La ventaja de **varargs** es que es más robusto, ya que inserta el separador de ruta del sistema operativo adecuado por ti (sin tener que poner `/` o `\`).

`Paths.get()`

El método `Path.of()` se introdujo en Java 11. Otra forma de obtener una instancia de `Path` es desde la clase Factory `java.nio.file.Paths` (empleada para crear objetos). Ten en cuenta la 's' al final de la clase `Paths` para distinguirla de la interfaz `Path`. Podemos reescribir los ejemplos anteriores usando [Paths.get\(\)](#):

```
Path path1c = Paths.get("fotos/batman.png");
Path path2c = Paths.get("c:\\users\\usuarioa\\notas.txt");
Path path3c = Paths.get("/", "home", "usuarioa");
```

`Paths.get()` es más "antiguo", pero **puede usarse de manera totalmente intercambiable con `Paths.get()`**.

Usando `java.net.URI`

Otra forma de **construir un `Path` con los métodos anteriores es proporcionando un valor de URI**. Un *identificador uniforme de recursos* (URI) es una cadena de caracteres que identifica un recurso (remoto o local). Comienza con un esquema que indica el tipo de recurso, seguido de un valor de ruta. Ejemplos de valores de esquema incluyen `file://` para sistemas de archivos locales, y `http://`, `https://` y `ftp://` para sistemas de archivos remotos.

La clase [java.net.URI](#) se utiliza para crear valores de URI.

Java incluye varios métodos Factory para la conversión entre objetos `Path` y `URI`, creación de `Path` y creación de `URI`.

Los siguientes ejemplos hacen referencia al mismo archivo (ojo no está implantado para `http` y `https`, en principio):

```
try {
    URI uri = new URI("file:///home/usuarioa/Prueba.txt");
    System.out.println(uri.isAbsolute());
    Path pathUri = Path.of(uri); // Creación de Path a partir de URI.
    Path pathUri2 = Paths.get(uri); // Creación de Path a partir de URI.
    URI uri2 = pathUri.toUri(); // Conversión de un Path en URI.
} catch (URISyntaxException e) { // Posible Excepción!!
}
```

Algunos de estos ejemplos **pueden lanzar una `IllegalArgumentException` en tiempo de ejecución**, ya que **algunos sistemas requieren que los URI sean absolutos**. La clase

URI tiene un método `isAbsolute()`, aunque se refiere a si el URI tiene un esquema, no a la ubicación del archivo.

Usando `FileSystem.getPath()`

La clase `FileSystems` crea instancias de la clase abstracta `FileSystem`.

```
FileSystem fileSystem = FileSystems.getDefault();
```

La clase `FileSystem` incluye métodos para trabajar directamente con el sistema de archivos. De hecho, tanto `Paths.get()` como `Path.of()` son en realidad atajos para el método `FileSystem.getPath()`:

```
Path path1d = fileSystem.getPath("fotos/batman.png");
Path path2d = fileSystem.getPath("c:\\users\\usuarioa\\notas.txt");
Path path3d = fileSystem.getPath("/home/usuarioa");
```

Conexión a Sistemas de Archivos Remotos

Si bien la mayor parte del tiempo queremos acceso a un objeto `Path` que esté dentro del sistema de archivos local, la clase `FileSystems` nos permite conectarnos a un sistema de archivos remoto con el método `FileSystems.getFileSystem()`:

```
fileSystem = FileSystems.getFileSystem(new URI("https://www.imdb.com"));
Path pathImdb = fileSystem.getPath("top250.txt");
```

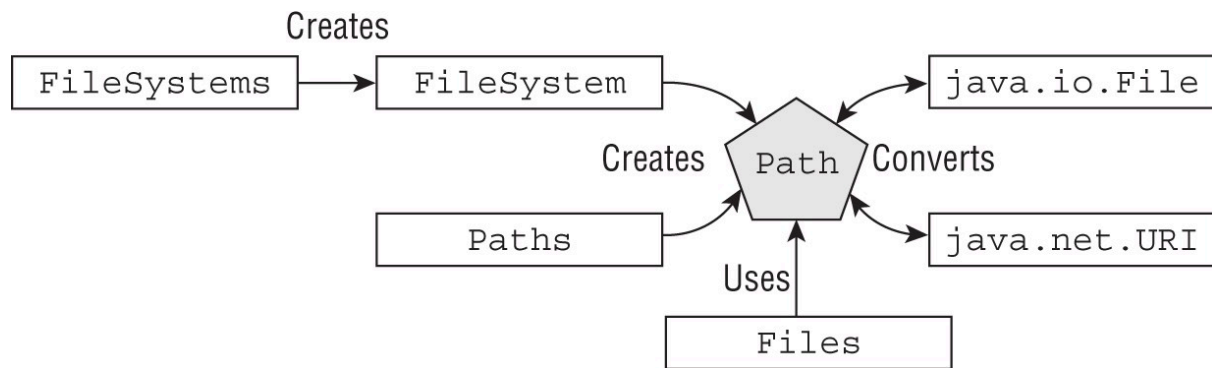
Este código es útil cuando necesitamos construir objetos `Path` con frecuencia para un sistema de archivos remoto. NIO.2 nos permite conectarnos tanto a sistemas de archivos locales como remotos, lo cual es una mejora importante sobre la antigua clase `java.io.File`.

Conversión de `java.io.File` a `java.nio.file.Path` y viceversa.

```
Path path = file.toPath(); // De File a Path
```

```
File file2 = path.toFile(); // De Path a File
```

Relaciones de la clase `java.nio.file.Path`.



Métodos comunes de Path

Al igual que los valores de String, entre otros objetos, las **instancias de Path son inmutables**.

```
Path p = Path.of("ballena");
p.resolve("krill"); // Se pierde, debería guardarse en otro Path.
System.out.println(p); // ballena
```

Muchos de los métodos disponibles en la interfaz Path **transforman de alguna manera el valor del path y devuelven un nuevo objeto Path, permitiendo encadenar los métodos**.

```
Path.of("/zoo/../../home").getParent().normalize().toAbsolutePath();
```

- `getFileName()`
- `getParent()`
- `getRoot()`
- `toString()`
- `getNameCount()`
- `getName()`
- `isAbsolute()`
- `toAbsolutePath()`
- `toRealPath()`
- `resolve(Path other)`
- `relativize(Path other)`
- `normalize()`
- `subpath()`

El primer método, `toString()`, devuelve una representación de cadena del path completo. De hecho, es el único método en la interfaz Path que devuelve una cadena. Muchos de los otros métodos en la interfaz Path devuelven instancias de Path.

Los métodos `getNameCount()` y `getName()` se usan a menudo en conjunto para recuperar el número de elementos en la ruta y una referencia a cada elemento,

respectivamente. Estos dos métodos **no incluyen el directorio raíz como parte del path**.

```
Path path = Paths.get("/tierra/hipopotamo/harry.feliz");

System.out.println("El nombre del Path es: " + path);

for(int i=0; i<path.getNameCount(); i++) {

    System.out.println(" Elemento " + i + " es: " + path.getName(i));

}
```

Aunque esta es una ruta absoluta, el elemento raíz no se incluye en la lista de nombres. Como dijimos, estos métodos no consideran el directorio raíz como parte del path.

```
var p = Path.of("/");

System.out.print(p.getNameCount()); // 0

System.out.print(p.getName(0)); // IllegalArgumentException
```

Observa que si intentas llamar a getName() con un índice no válido, lanzará una excepción en tiempo de ejecución.

Argumentos Opcionales en métodos de NIO.2

Muchos de los métodos de java NIO.2 **incluyen un varargs que toma una lista opcional de valores**. En la siguiente tabla se presentan argumentos comunes:

Tipo de Enum	Interfaz Heredada	Valor de Enum	Detalles
LinkOption	CopyOption, OpenOption	NOFOLLOW_LINKS	No seguir enlaces simbólicos.
StandardCopyOption	CopyOption	ATOMIC_MOVE	Mover archivo como operación atómica del sistema de archivos.
		COPY_ATTRIBUTES	Copiar atributos existentes al nuevo archivo.

		REPLACE_EXISTING	Sobrescribir el archivo si ya existe.
StandardOpenOption	OpenOption	APPEND	Si el archivo ya está abierto para escribir, entonces añadir al final.
		CREATE	Crear un nuevo archivo si no existe.
		CREATE_NEW	Crear un nuevo archivo solo si no existe, fallar en caso contrario.
		READ	Abrir para acceso de lectura.
		TRUNCATE_EXISTING	Si el archivo ya está abierto para escribir, entonces borrar el archivo y añadir al principio.
		WRITE	Abrir para acceso de escritura.

```
Path path = Paths.get("schedule.xml");
boolean exists = Files.exists(path, LinkOption.NOFOLLOW_LINKS);

Files.move(source, target, LinkOption.NOFOLLOW_LINKS,
StandardCopyOption.ATOMIC_MOVE);
```

Excepciones

Muchos de los métodos presentados en este apartado lanzan (pueden lanzar) una `IOException`. Las causas comunes de que un método lance esta excepción incluyen:

- **Pérdida de comunicación con el sistema de archivos** subyacente.
- El archivo o directorio existe pero **no se puede acceder o modificar**. El archivo existe pero **no se puede sobrescribir**.
- Se requiere el archivo o directorio pero **no existe**.

En general, los métodos que operan en valores abstractos de `Path`, como los de la interfaz `Path` o la clase `Paths`, a menudo no lanzan ninguna excepción verificada. Por otro lado, los métodos que operan o cambian archivos y directorios, como los de la clase `Files`, a menudo declaran `IOException`.

Hay excepciones a esta regla, como veremos. Por ejemplo, el método `Files.exists()` no declara `IOException`. Si lanzara una excepción cuando el archivo no existiera, ¡nunca podría devolver `false`!

La clase Files

La clase [Files](#) contiene métodos estáticos para realizar casi todas las operaciones con archivos y directorios, de forma:

- Segura: controla errores y permisos, evitando comportamientos inesperados.
 - Lanza excepciones claras (NoSuchFileException, AccessDeniedException) en lugar de fallar silenciosamente.
 - Comprueba permisos antes de leer o escribir.
 - Evita corrupción de datos al manejar flujos y buffers correctamente.
- Atómica: se realiza completamente o no se realiza en absoluto. No queda a medias aunque ocurra un fallo o se interrumpa el proceso. Esto es clave para evitar inconsistencias en sistemas críticos (bases de datos, backups, etc.).
- Multiplataforma: el mismo código funciona en distintos sistemas operativos gracias a la abstracción de Java, es decir, funciona en Windows, Linux y macOS sin cambios. Java abstrae las diferencias:
 - Convierte automáticamente separadores de ruta (/ o \).
 - Usa el FileSystem por defecto del sistema.
 - Adapta internamente permisos y atributos según el SO.

Métodos:

- para Verificación y Metadatos
- para Creación
- para Manipulación
- para Lectura y Escritura
- para Listado y Recorrido

Programación funcional. Algunos métodos de la clase Files utilizan la [API Stream de Java](#) que permite utilizar programación funcional para procesar colecciones de datos de manera declarativa mediante operaciones como filter, map y reduce.

- find()
- lines()
- list()
- walk()
- newDirectoryStream()

Los métodos de la API de Streams de NIO.2 utilizan una búsqueda en profundidad con un límite de profundidad, que puede cambiarse opcionalmente.

Los streams utilizan la evaluación “perezosa”, esto significa que los métodos cargarán en memoria sólo la información necesaria en cada ocasión.

Para cerrar automáticamente los Streams a archivos colocamos los objetos Stream dentro de un bloque try-with-resources

Repite los ejercicios **1, 2 y 3** utilizando la API java.nio ([Path](#), [Files](#)). Añade el sufijo NIO al nombre de clase.

Ejercicio. Realiza un programa que copie todos los archivos *.java (incluidos subdirectorios) en un directorio destino.

Si el directorio destino no existe debe crearlo.
Filtra de modo que el nombre del archivo termine en .java

Conversión entre java.io.File y java.nio.file

La **clase** java.io.File **proporciona el método** [toPath\(\)](#), **que convierte una instancia de estilo antiguo en una instancia** java.nio.file.Path:

```
Path path = file.toPath(); // De File a Path
```

De esta forma, puedes aprovechar el conjunto de características que ofrece la clase Path.

Por ejemplo, si tuvieras algún código que eliminara un archivo:

```
file.delete(); // Borrar con java.io
```

Podrías modificar este código para usar el método Files.delete, de la siguiente manera:

```
try {  
    Files.delete(path); // Borrar con java.nio (Posibles excepciones)  
} catch (IOException e) {  
}
```

A la inversa, el **método** [Path.toFile](#) **construye un objeto** java.io.File **para un objeto** Path.

```
File file2 = path.toFile(); // De Path a File
```

Mapecto de la Funcionalidad de java.io.File a java.nio.file

No hay una correspondencia uno a uno entre las dos APIs, pero la siguiente tabla da una idea general de qué funcionalidad en la API java.io.File corresponde a la funcionalidad en la API java.nio.file, y te indica dónde puedes obtener más información.

Funcionalidad de java.io.File	Funcionalidad de java.nio.file	Uso
java.io.File	java.nio.file.Path	Clase principal de gestión de archivos.

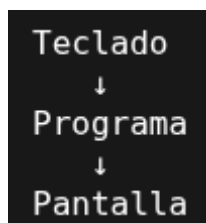
<code>java.io.RandomAccessFile</code>	<code>SeekableByteChannel</code>	Archivos de Acceso Aleatorio
<code>File.canRead</code> <code>File.canWrite</code> <code>File.canExecute</code>	<code>Files.isReadable</code> , <code>Files.isWritable</code> , <code>Files.isExecutable</code>	Verificación de permisos
<code>File.isDirectory()</code> <code>File.isFile()</code> <code>File.length()</code>	<code>Files.isDirectory(Path, LinkOption...)</code> , <code>Files.isRegularFile(Path, LinkOption...)</code> , <code>Files.size(Path)</code>	Gestión de Metadatos de archivo/directorio.
<code>File.lastModified()</code> , <code>File.setLastModified(long)</code>	<code>Files.getLastModifiedTime(Path, LinkOption...)</code> , <code>Files.setLastModifiedTime(Path, FileTime)</code>	Gestión de Metadatos de fecha modificación.
Métodos que establecen varios atributos (<code>setExecutable</code> , <code>setReadable</code> , <code>setReadOnly</code> , <code>setWritable</code>)	<code>Files.setAttribute(Path, String, Object, LinkOption...)</code>	Gestión de Metadatos de atributos de archivo.
<code>new File(parent, "newfile")</code>	<code>parent.resolve("newfile")</code>	Operaciones con archivos
<code>File.renameTo</code>	<code>Files.move</code>	Mover un Archivo o Directorio
<code>File.delete</code>	<code>Files.delete</code>	Eliminar un Archivo o Directorio
<code>File.createNewFile</code>	<code>Files.createFile</code>	Crear Archivos
<code>File.deleteOnExit</code>	Opción <code>DELETE_ON_CLOSE</code> especificada en <code>createFile</code>	Borrado de archivos al salir.
<code>File.createTempFile</code>	<code>Files.createTempFile(Path, String, FileAttributes<?>)</code> , <code>Files.createTempFile(Path, String, String, FileAttributes<?>)</code>	Crear Archivos temporales.

<code>File.exists</code>	<code>Files.exists, Files.notExists</code>	Verificar la Existencia de un Archivo o Directorio
<code>File.compareTo, equals</code>	<code>Path.compareTo, equals</code>	Comparar dos archivos/paths
<code>File.getAbsolutePath, getAbsoluteFile</code>	<code>Path.toAbsolutePath</code>	Obtención de la ruta absoluta.
<code>File.getCanonicalPath, getCanonicalFile</code>	<code>Path.toRealPath</code> o <code>normalize</code>	Convertir un Path (<code>toRealPath</code>), Eliminar Redundancias en un Path (<code>normalize</code>)
<code>File.toURI</code>	<code>Path.toURI</code>	Convertir un path en una URL.
<code>File.isHidden</code>	<code>Files.isHidden</code>	Saber si está oculto.
<code>File.list, listFiles</code>	<code>Path.newDirectoryStream</code>	Listar el Contenido de un Directorio
<code>File.mkdir, mkdirs</code>	<code>Files.createDirectory</code>	Crear directorio/s
<code>File.listRoots</code>	<code>FileSystem.getRootDirectories</code>	Listar los Directorios Raíz del Sistema de Archivos
<code>File.getTotalSpace, getFreeSpace, getUsableSpace</code>	<code>FileStore.getTotalSpace, getUnallocatedSpace, getUsableSpace, getTotalSpace</code>	Atributos del Almacenamiento de Archivos

3. E/S estándar

En Java se accede a la E/S estándar a través de los atributos estáticos de la clase `java.lang.System`

- Es una clase final y todos sus contenidos son privados.
- No se puede instanciar un objeto de esa clase.
- La clase `System` siempre está ahí disponible para que se pueda invocar cualquiera de sus métodos.
- Todos los miembros del paquete `java.lang` se pueden usar directamente, no hay que importarlos.
- Contiene tres variables con flujos predefinidos llamadas **in**, **out** y **err**, que se pueden utilizar sin tener una referencia a un objeto `System` específico.
 - **System.in**: implementa la entrada estándar. Por defecto es el teclado.
 - Instancia de la clase `InputStream`: flujo de bytes de entrada
 - `read()`: permite leer un byte de la entrada como entero
 - `skip(n)`: ignora `n` bytes de la entrada como entero
 - `available()`: número de bytes disponibles para leer en la entrada
 - **System.out**: implementa la salida estándar. Por defecto es la pantalla.
 - Instancia de la clase `PrintStream` (subclase de `OutputStream`): flujo de bytes de salida
 - Métodos para impresión de datos: `print()`, `println()`
 - `flush()`: vacía el buffer de salida escribiendo su contenido
 - **System.err**: implementa la salida de error. Por defecto es la pantalla.
 - Funcionamiento similar a `System.out`
 - Se utiliza para enviar mensajes de error (por ejemplo a un fichero de log o a la consola)
- El sistema se encarga de abrir y cerrar esos flujos del sistema automáticamente.



Exemplo de programa que usa a E/S estándar. Inclúe

- `System.in`: entrada estándar.
- `System.out`: saída estándar.
- `System.err`: saída de erros.
- `Scanner`: lectura de datos.
- conversión de texto a tipos primitivos.
- validación de datos.
- tratamiento básico de erros.

```
Scanner sc = new Scanner(System.in);  
  
System.out.println("=== XESTOR DE PRODUTOS ===");
```

```

System.out.print("Nome do produto: ");
String nome = sc.nextLine();

System.out.print("Prezo: ");
double prezo = sc.nextDouble();

System.out.print("Unidades: ");
int unidades = sc.nextInt();

sc.close();

if (prezo < 0 || unidades < 0) {
    System.err.println("Erro: o prezo e as unidades non poden ser negativos.");
    return;
}

double total = prezo * unidades;

System.out.println();
System.out.println("=== RESUMO ===");
System.out.println("Produto: " + nome);
System.out.println("Prezo: " + prezo + " €");
System.out.println("Unidades: " + unidades);
System.out.println("Total: " + total + " €");

```

Propostas de exercicios de E/S

1. Datos persoais

Pedir por teclado:

nome;

apelidos;

idade;

cidade.

Mostrar posteriormente os datos nun formato ordenado.

2. Calculadora

Pedir dous números e unha operación (+, -, *, /) e mostrar o resultado.

Debe controlar a división por cero.

3. Clase **Produto**

Crear:

```

public class Produto {
    private int id;
    private String nome;
    private double prezo;
}

```

O programa debe pedir os datos por teclado, crear un Produto e mostrar os seus datos.

4. Lista de produtos

Pedir varios produtos e almacenalos nun:

`ArrayList<Produto>`

Ao finalizar, mostrar todos os produtos e o importe total.

5. Menú de Produtos

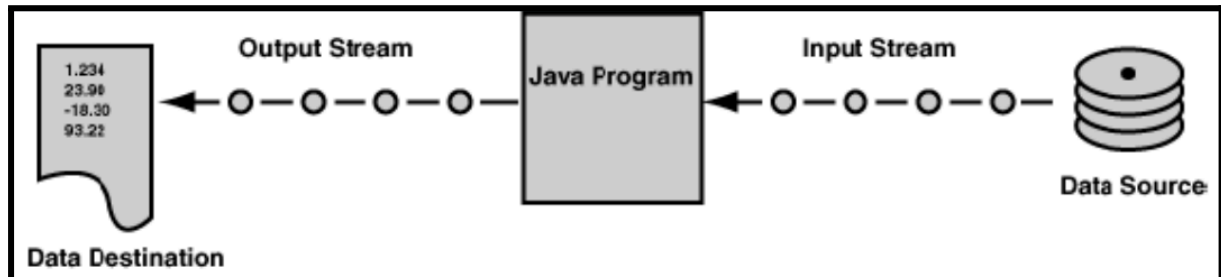
Crear un menú:

1. Engadir produto
2. Listar produtos
3. Buscar produto
4. Eliminar produto
5. Saír

Nota: Posteriormente retocaremos este programa para pasar de almacenar os datos en memoria a almacenalos nun ficheiro.

4. Flujos de Datos de E/S

Un flujo de datos (***stream***) es una **abstracción** de todo aquello que produzca o consuma información.



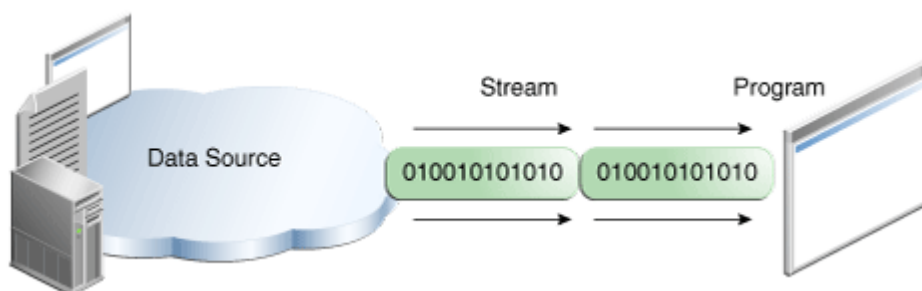
Todos los flujos se comportan de la misma manera, aunque traten con dispositivos distintos (ficheros, teclado, monitor, sockets de red, etc.). Así, **se pueden aplicar las mismas clases a cualquier tipo de dispositivo físico. La vinculación del *stream* al dispositivo físico lo lleva a cabo el sistema de entrada y salida de Java.**

La fuente de datos (*data source*) y el destino de los datos (*data destination*) de los diagramas anteriores puede ser cualquier cosa que genere o consuma datos. Esto incluye archivos de disco, un dispositivo (teclado, monitor, impresora, periférico, etc.), un *socket* de red (es un método para la comunicación entre un programa del cliente y un programa del servidor en una red) o un array.

En Java, los flujos de E/S se encuentran en el paquete `java.io`

Tipos de flujos

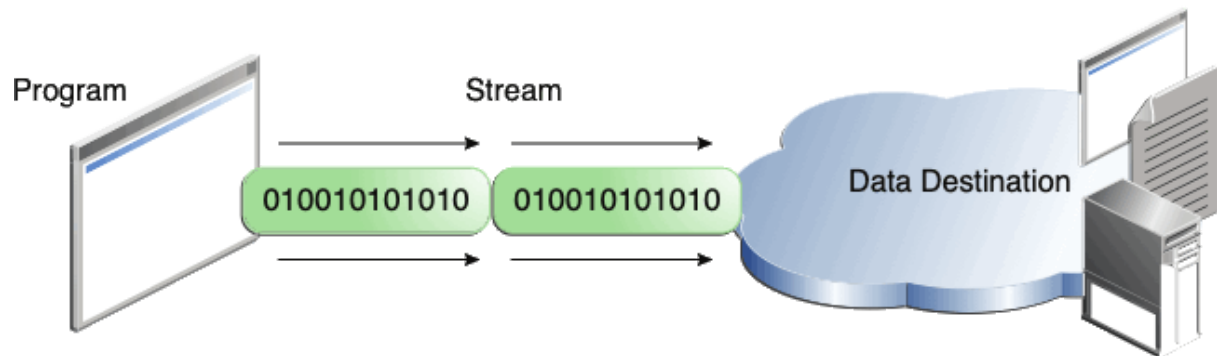
Flujo de entrada



Se utiliza un flujo de entrada (*input stream*) para obtener los datos leídos de una fuente. Pueden proceder de diferentes **tipos de fuentes**:

- **Archivos** de disco (`FileReader`, `FileInputStream`, `FileImageInputStream`,...).
- **Dispositivos**: teclado (`System.in`),...
- Otros **programas**.
- **Arrays** de memoria: `StringReader`

Flujo de salida



Se utiliza un flujo de salida (*output stream*) para escribir los datos a un destino. Puede representar diferentes **tipos de destinos**:

- **Archivos** de disco: `FileWriter`, `FileOutputStream`,...
- **Dispositivos**: pantalla (`System.out`),...
- Otros **programas**.
- **Arrays** de memoria: `ByteArrayOutputStream`,...

Cada tipo de flujo puede segmentar los datos de una manera particular. Por ejemplo, algunas clases de flujos leen o escriben datos como bytes individuales. Otras clases de flujos leen o escriben caracteres individuales o cadenas de caracteres. Además, **algunas clases de flujos leen o escriben grupos más grandes de bytes o caracteres a la vez, específicamente aquellas con la palabra “*Buffered*” en su nombre.**

Las clases [PrintWriter](#) y [PrintStream](#) representan flujos de salida que no tienen clases equivalentes que representen flujos de entrada. Su propósito es facilitar la escritura de datos formateados en un flujo, como sucede con [System.out](#) y [System.err](#) que son de tipo `PrintStream`.

Flujos de bytes

- **Leen/escriben datos binarios (0 y 1)**
- Las clases heredan de las clases abstractas [InputStream](#) y [OutputStream](#).
- Hay muchas clases de flujos de bytes, como: `FileInputStream` y `FileOutputStream`. Todos los restantes flujos funcionan del mismo modo sólo difieren en la forma de construirlos.

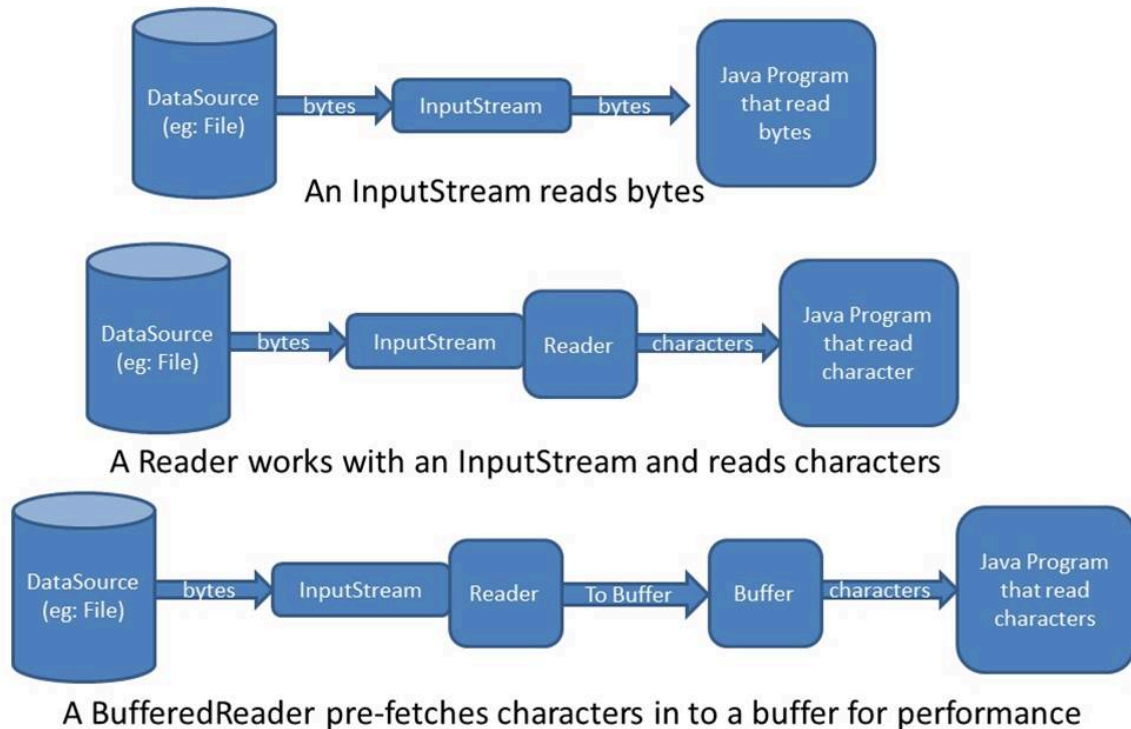
Investiga los métodos de las clases abstractas [InputStream](#) y [OutputStream](#) y sus subclases.

Flujos de caracteres

- **Leen/escriben datos de texto.**
- Transmiten caracteres de 16 bits codificados en Unicode.
- Al nivel más bajo todas las operaciones de I/O son orientadas a byte.

- Las clases heredan de las clases abstractas [Reader](#) y [Writer](#).
- Hay muchas clases de flujos de carácter, como `:FileReader` (usa internamente `FileInputStream`), `FileWriter` (usa internamente `FileOutputStream`). Todos los restantes flujos funcionan de igual modo, sólo difieren en la forma de construirlos.

Investiga los métodos de las clases abstractas [Reader](#) y [Writer](#) y sus subclases.



Usando flujos de datos

Las clases [InputStream](#), [OutputStream](#), [Reader](#) y [Writer](#) son clases abstractas.

Al tratarse de clases abstractas, no vamos a poder crear objetos de estas clases porque su funcionalidad está "incompleta" (tienen métodos que no están definidos, implementándolos las subclases), y así nos permiten representar un flujo de datos de entrada o salida binario cualquiera.

Las clases hijas proporcionan funcionalidades más específicas. Para usar flujos tendremos que crear objetos de estas subclases.

Las clases de `java.io` siguen una nomenclatura sistemática que permite deducir su función a partir de las palabras que componen el nombre, tal como se describe en la siguiente tabla:

Palabra	Significado
InputStream, OutputStream	Lectura/Escritura de bytes
Reader, Writer	Lectura/Escritura de caracteres

File	Archivos y directorios
Buffered	Buffer
Filter	Filtro sobre el stream
Data	Intercambio de datos en formato propio de Java (datos primitivos)
Object	Persistencia de objetos
RandomAccessFile	Acceso directo o aleatorio

Nota: Un **buffer** es un espacio de memoria intermedia. Cuando se necesita un dato del disco se trae a memoria ese dato y sus datos contiguos, de modo que la siguiente vez que se necesite algo del disco la probabilidad de que esté ya en memoria sea muy alta. Algo similar se hace para escritura, intentando realizar en una sola operación de escritura física varias sentencias individuales de escritura.

Clases principales:

Tipo de archivo	Lectura (Input)	Escritura (Output)	Modo de acceso
Binarios (bytes)	FileInputStream, BufferedInputStream, DataInputStream	FileOutputStream, BufferedOutputStream, DataOutputStream	Secuencial
Texto (caracteres)	FileReader, BufferedReader, InputStreamReader	FileWriter, BufferedWriter, PrintWriter	Secuencial
Objetos	ObjectInputStream	ObjectOutputStream	Secuencial
Acceso aleatorio	RandomAccessFile	RandomAccessFile	Aleatorio

El siguiente ejemplo utiliza flujos de bytes para hacer una copia de un archivo **copiando byte a byte**.

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class EjemploCopiaArchivos {

    public static void main(String[] args) throws IOException {
        FileInputStream in = null;
        FileOutputStream out = null;
        try {
```

```

        in = new FileInputStream("origen.txt");
        out = new FileOutputStream("destino.txt");
        int c;
        while ((c = in.read()) != -1) {
            out.write(c);
        }
    } finally { // Hay que cerrar el flujo en cualquier condición.
        if (in != null) {
            in.close();
        }
        if (out != null) {
            out.close();
        }
    }
}

```

- El método `read()` devuelve un valor de byte en forma de un entero, para poder emplear -1 como fin de flujo. Cuando se alcanza el final del archivo, `read()` devuelve -1.
- El método `write()` escribe un byte en el flujo de salida.
- El método `close()` cierra el flujo. Si no se cierra, el sistema operativo puede no liberar los recursos asociados con el archivo.
- Para **ficheros de texto (con caracteres, como en el ejemplo)** es mejor emplear **flujos de caracteres (character streams)**.

Cerrar un flujo cuando ya no se necesita es muy importante. `CopiaArchivos` utiliza un bloque `finally` para garantizar que ambos flujos se cierren incluso si se produce un error. Esta práctica ayuda a evitar graves pérdidas de recursos.

La técnica más recomendada es utilizar **try-with-resources**, que permite que los flujos se cierren automáticamente al final del bloque `try`.

Practica. Mejora el programa anterior:

1. Escribe código en respuesta a las posibles excepciones
2. Utiliza un `try-with-resources`
3. Utiliza un flujo *Buffered* para mejorar el rendimiento.
4. Permite al usuario seleccionar el fichero a copiar y el destino.

Flujos binarios

- [FileOutputStream](#) y [FileInputStream](#) permiten crear flujos de entrada salida para trabajar con ficheros binarios.

FirmasHexadecimales.java

Más allá de su extensión (.pdf, .jpg, .png, .doc, etc.), algunos formatos de archivo pueden reconocerse por los valores de sus primeros y últimos bytes, en lo que se conoce como firmas hexadecimales o "números mágicos". En la siguiente página puedes consultar algunos de estas firmas: https://garykessler.net/library/file_sigs.html

Elabora un programa que permita seleccionar archivos de al menos 3 extensiones distintas y que comprueba si la firma coincide con su extensión.

CifradoBinario.java

Crea un programa que cifre un fichero binario invirtiendo los valores de sus bits para hacerlo ilegible.

Comprueba que aplicando proceso de nuevo el fichero vuelve a ser legible.

Flujos de texto

- [FileReader](#) y [FileWriter](#) están orientados a ficheros de texto pero, en general, funcionan de forma similar a sus equivalentes de flujos de datos binarios.

AnalisisTexto.java

Crea un programa que solicite un fichero de texto y cuente el número de caracteres, de vocales, de consonantes, de dígitos y de espacios en blanco.

LineasNumeradas.java

Crea un programa que lea un fichero de texto y genere una copia en la que cada línea vaya precedida por el número de línea ("1: ", "2: ", "3: ", etc.)

ComparadorFicheros.java

Crea un programa que compare dos ficheros de texto e indique si son idénticos y, en caso de no serlo, en que línea y columna tienen el primer carácter distinto.

ParticipacionEnClase.java

...

Los flujos de salida `FileOutputStream` y `FileWriter` tienen una primera variante de constructor que creará o sobrescribirá el archivo indicado, si existe, y otra variante con un segundo parámetro (**append**) que en caso de valer true abrirá el fichero para escribir datos al final del mismo.

Flujos con Búfer

- [BufferedOutputStream](#) y [BufferedInputStream](#) permiten encapsular flujos para utilizar un buffer en las operaciones de lectura y escritura y así mejorar el rendimiento.
- [BufferedReader](#) y [BufferedWriter](#) aportan la misma funcionalidad pero aplicada a flujos de texto.

Los flujos de salida de tipo `Buffered` no escriben los datos en disco hasta que se llena el búfer. Es por eso que si el programa termina anormalmente podrían producirse pérdidas de

datos que aún no se hayan grabado. Para forzar la escritura de los datos del búfer podemos usar el método **flush()**. En cualquier caso los datos se escribirán en disco al cerrar el flujo.

ComparaRendimiento.java

Crea un programa que escriba un número de bytes introducido por el usuario (por defecto 1.000.000) de dos formas:

- a) con `FileOutputStream` directo,
- b) con `BufferedOutputStream`.

Mide el tiempo empleado por cada método, por ejemplo con `System.nanoTime()` y muestra el resultado de la comparación.

Amplía el experimento permitiendo al usuario elegir el tamaño del buffer y guardando en un fichero de texto (`PruebasRendimiento.txt`) las condiciones y los resultados de cada prueba.

LecturaPorLineas.java

Lee un fichero de texto con `BufferedReader.readLine()` y muestra por pantalla solo las líneas que contengan una palabra clave introducida por el usuario.

RegistroAlumnado.java

Genera un fichero `alumnos.txt` con 1000 líneas del tipo `Alumno_i;nota_i` donde la nota es aleatoria entre 0 y 10. Usa `BufferedWriter` y `newline()` para los saltos de línea.

EstadisticasAlumnado.java

Lee `alumnos.txt` (del ejercicio anterior) con `BufferedReader`, parsea cada línea, y calcula: nota media, máxima, mínima y cuántos aprobados hay. Muestra también el nombre del mejor alumno.

Flujos de objetos serializados

[ObjectInputStream](#) y [ObjectOutputStream](#)

Serialización de objetos

Flujos desde una URL

```
import java.io.*;
import java.net.URI;
import java.net.URL;

public class EjemploLeerURL {
    public static void main(String[] args) throws Exception {
        URI uri = new URI("https://data.iana.org/TLD/tlds-alpha-by-domain.txt");
        URL url = uri.toURL();
        InputStream is = url.openStream();
        InputStreamReader isr = new InputStreamReader(is);
        // es un puente de bytes a caracteres.
        int c;
        while ((c = isr.read()) != -1) {
            System.out.print((char) c);
        }
    }
}
```

URI/URL

url.openStream(); // Abreviatura de:
url.openConnection().getInputStream(); // openConnection() devuelve un objeto de tipo URLConnection.

URLConnection

HttpURLConnection

Permite añadir elementos específicos de HTTP, como el tamaño del contenido, o el tipo de archivo:

```
URL url = new URI("https://data.iana.org/TLD/tlds-alpha-by-domain.txt").toURL();
HttpURLConnection httpConnection = (HttpURLConnection) url.openConnection();
// Hereda de URLConnection
httpConnection.getInputStream();
httpConnection.setRequestMethod("HEAD");
long tamanho = httpConnection.getContentLengthLong();
```

Ejercicios propuestos.

Lectura de URL

Crea un programa que lea el contenido de una URL y lo muestre por pantalla.

Mejore el programa para que pida una URL y la guarde en un archivo en una carpeta seleccionada del disco mediante un JFileChooser.

¿Serías capaz de mostrar el tamaño del contenido de la URL? ¿Y que ponga la extensión adecuada al archivo?

Ayuda: Puedes emplear HttpURLConnection para obtener el tamaño del contenido y para obtener el Content-Type puedes emplear el método getContentType().

Lectura de URL con HttpURLConnection

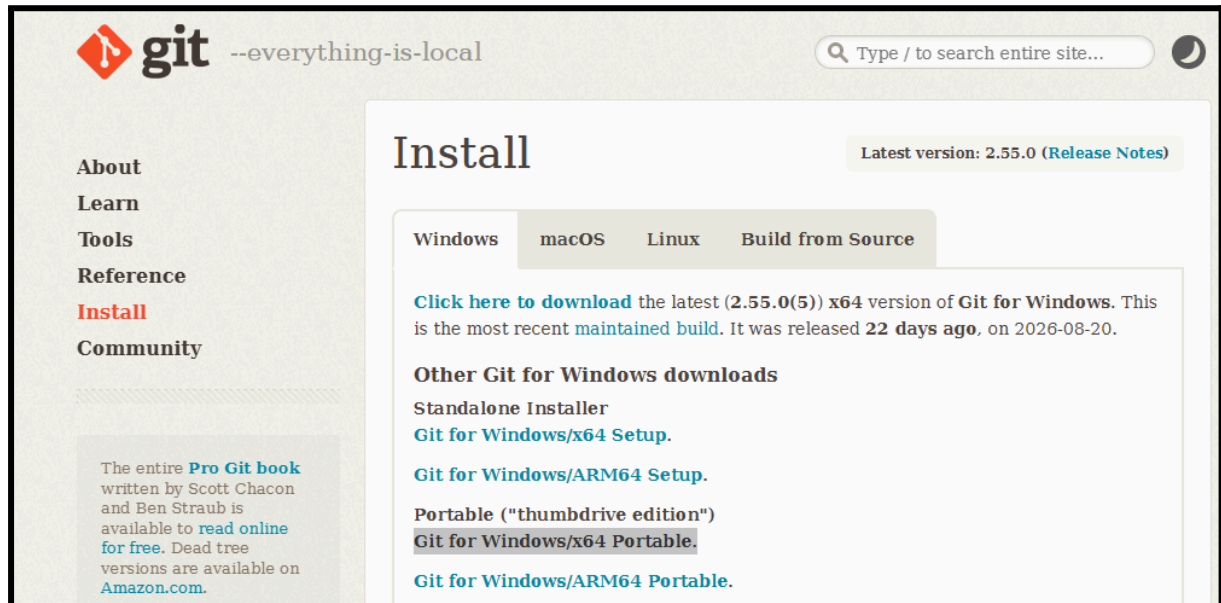
Amplía el ejercicio anterior para que emplee HttpURLConnection y muestre la información de la cabecera HTTP.

Anexo. Git

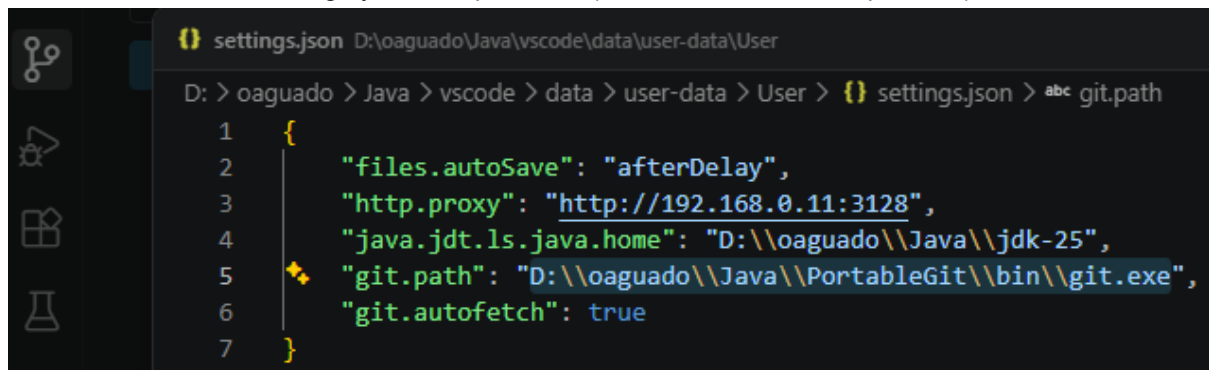
Recomendación: Utiliza [Git](#) en el centro y en casa para

1. gestionar tu código del curso ordenadamente.
2. acceder cómodamente al código compartido por el profesor.

- Página oficial de Git: <https://git-scm.com/>
- Repositorio del Profesor: <https://github.com/aguadodev/DAM2-AD-26-27>



Descarga y descomprime Git (en el centro la versión portable)



Configura el IDE (en la imagen vscode) para usar Git

Configuración inicial de Git

Ejecutar estos comandos de consola desde la carpeta del git.exe

Configurar usuario

A continuación podemos configurar o noso nome de usuario e mail. Ao facelo globalmente coa opción --global evítanos ter que facelo en cada proxecto. Se o nome ten espazos deberá ir entre comillas.

```
git config --global user.name "Oscar aguadodev"  
git config --global user.email aguado@edu.xunta.es
```

Configurar proxy en Git por comandos

No centro educativo pode ser necesario configurar o proxy para tarefas que requiran acceso a Internet. Podemos executar os comandos desde o Terminal de vscode.

```
git config --global http.proxy http://192.168.0.11:3128  
git config --global https.proxy http://192.168.0.11:3128
```